When to Think, What to Remember: Mixed-Initiative Hierarchical Robot Control with Procedural Memory

Yinpei Dai[1,2,†] **Jean Mercat**[2] **Isabella Huang**[2] **Shanmuga Harikumar**[2]
Paarth Shah[2] **Joyce Chai**[1] **Kushal Arora**[2,‡]
[1]University of Michigan [2]Toyota Research Institute
{yinpei.dai.pi, kushal.arora}@tri.global

Abstract: Hierarchical robot policies typically combine a vision-language model (VLM) for high-level reasoning with a vision-language-action model (VLA) for low-level control. However, these systems remain largely VLM-driven, with limited feedback from execution progress and little use of accumulated experience. We introduce MIRA (**Mi**xed-Initiative **R**easoning and **A**ction), a hierarchical framework that enables tighter coordination between reasoning, action, and memory. In MIRA, System 1 jointly predicts robot actions and subtask execution progress, allowing the VLA to help determine *when to think*. System 2 operates in three modes (planning, execution, and summary) to generate milestone-level plans and fine-grained subgoals, monitor task execution, and determine *what to remember* by distilling accumulated experience into reusable procedural memory. On RoboCasa365, MIRA improves the success rate of the $\pi_{0.5}$ baseline from 16.9% to as high as 53.7%. Furthermore, task recipes extracted from a single demonstration video improve one-shot success on unseen tasks from 3.5% to 16.7%, highlighting the benefits of coordinating reasoning, action, and memory.

1 Introduction

Vision-language-action (VLA) models have emerged as a promising route toward general-purpose robot policies that map visual observations and language goals directly to actions [1, 2]. Yet long-horizon manipulation requires more than reactive control: a robot must track what has happened and what remains to be done, determine when an ongoing behavior has completed, and distill reusable knowledge from prior experience. This naturally motivates hierarchical architectures, where a high-frequency VLA serves as a reactive System 1 and a lower-frequency vision-language model (VLM) acts as a deliberative System 2 [3, 4, 5, 6, 7]. Such hierarchies provide a natural interface between fast motor control and slower reasoning, but also raise two practical questions: *when should the system think, and what should it remember during and across episodes?*

Existing hierarchical systems remain largely *VLM-initiated*: System 2 generates a language subgoal, while System 1 passively executes it until the next high-level invocation. This handoff is typically governed by a fixed query frequency [3, 6], an external success detector [8, 9], or a VLM-predicted execution duration [9]. As a result, the low-level policy has limited ability to communicate its own execution progress or determine when further deliberation is needed. Meanwhile, repeatedly generating only the next command can bias System 2 toward shallow, reactive reasoning rather than maintaining a structured view of task progress and execution history, while also introducing redundant high-level queries. Beyond the current episode, cross-episode experience reuse remains comparatively underexplored. These limitations motivate a hierarchy that can both determine *when* deliberation is needed and preserve useful task knowledge within and across episodes.

[†]Work done during an internship at Toyota Research Institute.
[‡]Corresponding author.

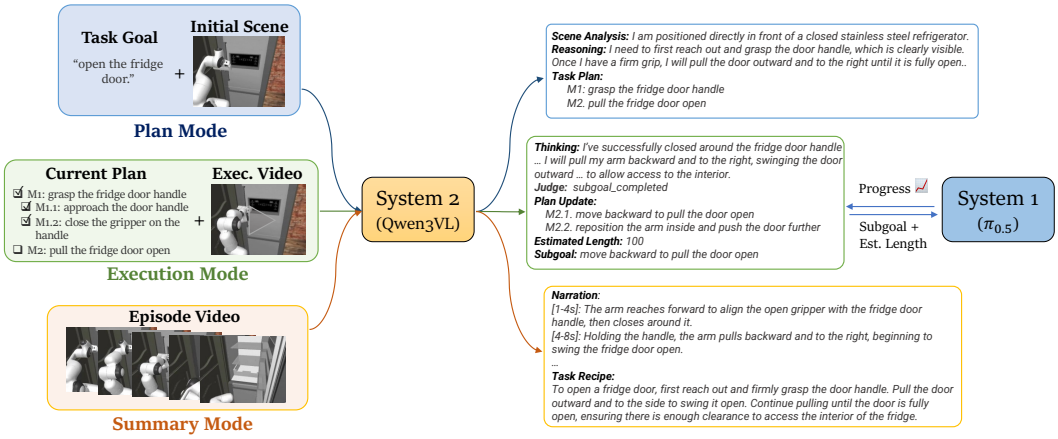


Figure 1: **Overview of MIRA.** System 2 operates in three modes: *Plan Mode* generates milestone plans, *Execution Mode* produces fine-grained subgoals and estimated execution length, and *Summary Mode* distills completed demonstrations into reusable task recipes. System 1 executes subgoals while predicting progress, enabling mixed-initiative communication between systems.

We propose MIRA (**Mi**xed-Initiative **R**easoning and **A**ction), a hierarchical robot policy that couples mixed-initiative control with procedural memory. The central idea is intuitive: *System 1 predicts actions and helps decide when to think, while System 2 determines what to reason about and what to remember*. Within an episode, *Plan Mode* constructs an active procedural memory as a milestone-level task plan, while *Execution Mode* maintains and updates its progress and decomposes the active milestone into fine-grained subgoals. Meanwhile, System 1 jointly predicts robot actions and normalized execution progress, allowing its execution state to trigger further deliberation. Across episodes, *Summary Mode* narrates a demonstration and consolidates it into a compact task recipe, providing long-term procedural memory for future planning. MIRA therefore maintains procedural knowledge both online during execution and across repeated task experiences.

To train these capabilities, we develop an automatic task segmentation and annotation pipeline that converts robot trajectories into milestone plans, fine-grained subgoals, narrations, progress supervision, and task recipes. We instantiate MIRA with $\pi_{0.5}$ [2] as System 1 and Qwen3-VL [10] as System 2. On RoboCasa365, the base MIRA system improves the $\pi_{0.5}$ baseline from 16.9% to 45.5% success, and lightweight language steering further raises performance to 53.7%. More importantly for memory-driven adaptation, on composite-unseen tasks, procedural memory extracted by System 2 from a single demonstration video improves one-shot success from 3.5% to 9.6%, and reaches 16.7% when combined with language steering, all without parameter updates.

In summary, our main contributions are:

- A mixed-initiative System 1/System 2 hierarchy where System 1 predicts actions and execution progress to help determine *when* deliberation should resume;
- A multi-mode System 2 for planning, execution, and summary, maintaining within-episode task structure and reusable cross-episode procedural memory;
- An automatic hierarchical annotation pipeline and experiments demonstrating strong long-horizon control and one-shot experience reuse on RoboCasa365.

Hierarchical vision-language-action models. Large vision-language-action models (VLAs) map observations and instructions directly to robot actions across diverse tasks [1, 2, 6]. Hierarchical approaches augment these policies with language-based reasoning. SayCan [11] selects skills using learned affordances, Inner Monologue [8] incorporates execution feedback into reasoning, and RT-H [12] predicts intermediate language motions. More recent methods pair VLM supervisors with robot policies. RACER [13] guides waypoint-based control, while Hi-Robot [3] generates language

commands for $\pi_{0.5}$ [2] and supports free-form human interaction. RoboCerebra [4] monitors execution and revises subgoals. In these systems, high-level reasoning largely determines when to intervene. MIRA extends this hierarchy with mixed-initiative control, allowing the VLA to predict execution progress and trigger event-driven handoffs to the VLM.

Memory and experience reuse. Robot memory supports both within-episode execution and reuse of prior experience. MemoryVLA [14] maintains perceptual and cognitive memory banks, MemER [15] retrieves task-relevant keyframes, and MEM [5] combines short-term visual memory with long-term textual memory. RoboMME [16] benchmarks these capabilities and provides memory-augmented $\pi_{0.5}$ baselines. Beyond individual episodes, RICL [17] retrieves relevant demonstration segments, while ViVLA [18] and GEN-1.5 [19] condition action policies directly on demonstrations. MIRA tracks execution through an explicit milestone plan and distills demonstrations into reusable language-based task recipes. These recipes guide high-level planning without requiring the low-level policy to process demonstration videos or actions.

Trajectory segmentation and annotation. Training hierarchical policies requires decomposing demonstrations into meaningful subtasks. MimicGen [20] identifies object-centric boundaries using task-specific signals, while UVD [21] detects boundaries through changes in pretrained visual representations. Recent methods enrich these segments with semantic supervision. RoboAnnotatorX [22] annotates long-horizon demonstrations, RoboInter [23] provides dense intermediate representations, IVLR [24] augments UVD segments with interleaved reasoning traces, and Cortex [25] organizes annotations around executable skills. Our pipeline combines visual-semantic boundaries with gripper and end-effector motion cues, then uses a VLM to consolidate

2 Hybrid Trajectory Segmentation and Annotation

Long-horizon manipulation tasks can be decomposed into a hierarchy of *milestones* (e.g., *"open the fridge door"*), each consisting of finer-grained *subgoals* (e.g., *"grasp the door handle"* and *"pull the door open"*). Training a hierarchical policy requires milestone plans and temporally aligned subgoal annotations, which are typically absent from raw robot demonstrations. We therefore develop an automatic two-stage pipeline (Fig. 2) that first identifies candidate subgoal boundaries using complementary robot-state and visual cues, then employs a VLM to merge semantically coherent clips and generate hierarchical annotations.

Candidate boundary detection. We identify candidate boundaries from three complementary sources. First, *coarse state boundaries* are detected at gripper opening and closing events of either arm, capturing transitions associated with grasping and releasing objects (①). Second, *fine state boundaries* are identified from transitions between moving and near-stationary end-effector states within each coarse segment, capturing finer motion phases such as reaching, transport, and alignment (②). Third, *semantic boundaries* are detected using UVD [21] within each coarse segment, capturing changes in visual representations that may not be apparent from robot-state signals alone (③). We combine these boundaries and consolidate near-duplicates to produce an over-segmentation of each demonstration into short clips (④). This stage favors boundary coverage over semantic precision, leaving potentially redundant boundaries for subsequent refinement.

Semantic merging and annotation. Given the task instruction, initial scene, and sequence of candidate clips, a VLM annotator (Gemini 3.1 Pro) identifies adjacent clips that belong to the same semantic action and merges them accordingly (⑤). Rather than predicting temporal boundaries from scratch, the VLM refines the proposed segmentation and assigns semantic labels to the resulting clips. Specifically, it generates an initial-scene description, a milestone-level plan, fine-grained subgoals aligned with the merged clips, and clip-level narrations. Each milestone corresponds to a contiguous sequence of subgoals, establishing temporal alignment between the two levels of the hierarchy. We additionally assign each subgoal a progress target that increases linearly from 0 to 1 over its duration. These annotations provide supervision for both levels of MIRA (Sec. 3): subgoal

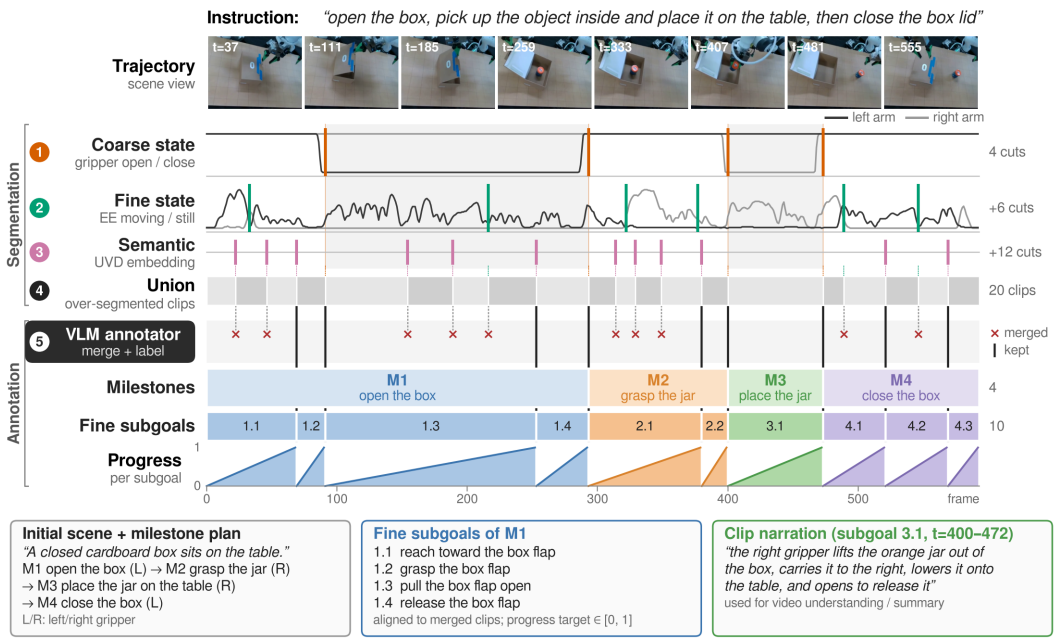


Figure 2: **Hybrid trajectory segmentation and annotation.** Illustration on a bimanual manipulation demonstration (593 frames). *Segmentation:* ① Gripper events define coarse boundaries; ② end-effector motion transitions introduce finer boundaries; and ③ UVD identifies additional visual-semantic boundaries. ④ Their union produces 20 candidate clips. *Annotation:* ⑤ A VLM merges semantically coherent clips (discarded boundaries marked by ×), retaining 9 of 19 candidate boundaries. The resulting 10 subgoals are organized into 4 milestones, with linear progress targets assigned to each subgoal. The VLM also generates an initial-scene description, milestone plan, and clip-level narrations (bottom).

progress targets train System 1 to estimate execution progress, while milestone plans, subgoals, narrations, and task recipes provide training examples for System 2 (Appendix A.1).

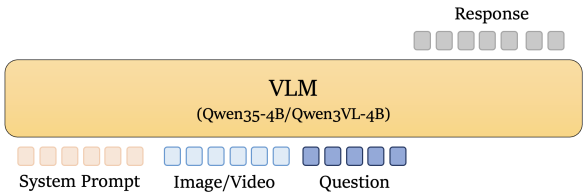
3 MIRA: Mixed-Initiative Reasoning and Action

We introduce MIRA, a hierarchical robot policy that coordinates high-level reasoning, low-level control, and experience reuse through two interacting systems. Given a task goal g, the robot receives multi-view observations o_t and proprioceptive state s_t, and predicts actions a_t. As illustrated in Fig. 1, MIRA consists of a deliberative System 2 that maintains a hierarchical task plan and a reactive System 1 that executes fine-grained subgoals. Unlike conventional VLM-driven hierarchies, MIRA allows System 1 to initiate reasoning based on its own execution progress, rather than relying solely on high-level decisions or fixed intervention intervals.

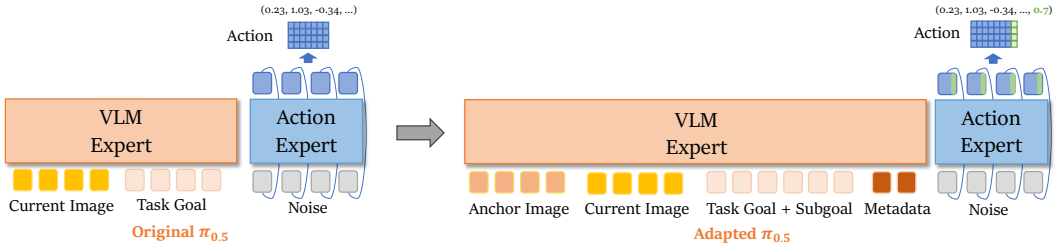
At the beginning of an episode, System 2 generates a milestone-level plan from the task goal and initial observations, optionally incorporating a task recipe extracted from a previous demonstration. It then selects a fine-grained subgoal z and estimates its execution length $\hat{L}$. Conditioned on this subgoal, System 1 predicts robot actions together with execution progress $p_t \in [0, 1]$:

$$(a_t, p_t) = \pi_{\mathrm{S1}}\big(o_t, o^{\mathrm{anc}}, s_t, g, z, \hat{L}\big), \tag{1}$$

where o^{anc} is an anchor observation captured at the beginning of the current subgoal. When progress or motion signals indicate that execution should be reassessed, System 1 hands control back to System 2, which evaluates the outcome, updates the plan, and issues the next subgoal. This cycle continues until System 2 determines that the task is complete. In this way, System 1 helps determine *when to think*, while System 2 determines *what to do* and *what to remember*.



(a) System 2: one VLM with three modes, all formulated as visual QA problems.



(b) System 1: $\pi_{0.5}$ (left) adapted to predict progress (right).

Figure 3: **Model architectures of MIRA.** (a) System 2 uses a single VLM shared across Plan, Execution, and Summary Modes. Each mode follows a unified visual question-answering format with a mode-specific system prompt, image or video observations, and a textual query, producing structured reasoning outputs. (b) System 1 extends $\pi_{0.5}$ with subgoal-conditioned inputs, including an anchor image and execution metadata. Its flow-matching action expert jointly predicts robot actions and subgoal progress, enabling progress-aware handoffs to System 2.

3.1 System 2: Multi-Mode Reasoning

System 2 is instantiated as a VLM operating in three modes: *Plan*, *Execution*, and *Summary*. As shown in Fig. 3a, all three modes share a visual question-answering formulation, with a mode-specific system prompt, visual context, and a textual query. Plan Mode processes the initial multi-view image, whereas Execution and Summary Modes process video clips. The model generates structured responses using tags such as `<plan>`, `<judge>`, and `<subgoal>`, which are parsed into the corresponding outputs. This unified formulation allows the three reasoning functions to share model parameters and be trained jointly. More details can be found in in Appendix A.2.

Plan Mode: Milestone planning. Plan Mode takes the task goal and initial scene as input and generates a milestone-level plan describing the major stages needed to complete the task. Each milestone represents a meaningful task outcome, such as opening a container or placing an object, rather than an individual robot motion. The plan is maintained as explicit working memory throughout execution, allowing subsequent decisions to track completed milestones and remaining steps. When a task recipe from a previous demonstration is available, Plan Mode additionally uses it to guide the plan for the current scene.

Execution Mode: Subgoal selection and monitoring. Execution Mode translates milestone plans into executable subgoals and monitors their completion. It is invoked immediately after planning and whenever System 1 requests a handoff. Given the task goal, current plan, robot status, and most recent execution clip, it performs three functions: (1) assesses the previous subgoal as `completed`, `incomplete`, or `failed`; (2) updates the milestone checklist and expands the active milestone into finer-grained subgoals; and (3) selects the next subgoal z together with an estimated execution length $\hat{L}$, discretized into 16 bins. By maintaining and updating the same plan across handoffs, Execution Mode preserves long-horizon task context while adapting subsequent actions to observed outcomes.

Summary Mode: Experience distillation. Beyond online task execution, Summary Mode converts demonstration videos into reusable procedural memory. Given a segmented demonstration, it generates a narration for each clip conditioned on the preceding narrations, then summarizes the

trajectory into a compact task recipe. The recipe captures procedural information such as action order, relevant object relationships, preconditions, and arm choices. During subsequent tasks, this recipe is provided to Plan Mode to guide milestone planning. Unlike approaches that directly condition action generation on past demonstrations, MIRA transfers experience through language-level planning, without requiring System 1 to process demonstration videos or actions.

3.2 System 1: Progress-Aware VLA

We instantiate System 1 by extending $\pi_{0.5}$ [2], which consists of a VLM expert for visual-language encoding and a flow-matching action expert for generating robot action chunks. As illustrated in Fig. 3b, we retain the original architecture while modifying its inputs and outputs to support subgoal-conditioned control and progress prediction.

Subgoal-conditioned inputs. In addition to the current observations and task goal, System 1 receives two forms of subgoal-specific context. First, an *anchor image* captured at the start of the current subgoal provides a visual reference for tracking changes during execution. Second, the language prompt includes the subgoal instruction and execution metadata, including the estimated length $\hat{L}$, the robot state at subgoal initiation, the current state, and gripper status. Together, these inputs provide both the intended behavior and the temporal context needed to estimate its progress. Prompt details can be found in in Appendix A.3.

Joint action and progress prediction. We augment each action in the predicted chunk with an additional scalar representing subgoal execution progress. The action expert therefore jointly predicts robot actions and progress through the same flow-matching process, without introducing additional prediction heads or training objectives. Following Sec. 2, progress supervision is defined by a linear increase from 0 to 1 over each annotated subgoal segment. Through joint training, System 1 learns to estimate how far execution has advanced while generating the corresponding actions. At inference time, these predictions provide an explicit signal for deciding when to return control to System 2.

3.3 Mixed-Initiative Handoff and Language Steering

Progress-aware handoff. Rather than invoking System 2 at fixed intervals, MIRA allows System 1 to initiate a handoff based on three conditions: (1) predicted subgoal progress approaches completion, (2) end-effector commands remain near zero without gripper events, indicating convergence or stalled execution, or (3) execution exceeds a timeout proportional to the estimated subgoal length $\hat{L}$. The first two conditions require sustained evidence over multiple steps to avoid premature handoffs (see Appendix A.4). A handoff does not imply success; instead, System 2 evaluates the execution clip and determines whether to advance, retry, or revise the current subgoal. This enables System 1 to determine *when to think*, while System 2 decides *how to proceed*.

Language steering. The language interface between System 1 and System 2 also enables lightweight corrections without retraining either model. We introduce heuristic steering rules that modify the subgoals generated by System 2 in response to common execution failures. For example, the system may request a regrasp after an unsuccessful grasp, repeat a button press when its effect is uncertain, or retry the current milestone after repeated failures. These interventions operate entirely through language instructions, allowing high-level reasoning to redirect low-level behavior without directly modifying the action predictions

4 Experiments

We first assess annotation quality, then investigate three questions: (RQ1) whether mixed-initiative coordination improves long-horizon manipulation, (RQ2) whether language steering enables test-time correction, and (RQ3) whether a single video provides reusable procedural memory.

Method	Miss.↓	Gran.↓	Broad↓	Clean↑
Uniform	24	**4**	37	52
Weighted	32	9	30	53
UVD	39	17	**21**	49
Ours	**21**	11	**21**	**61**

(a) Plan quality

Method	Corr.↑	Partial↓	Extra↓	Aligned↑
Uniform	83	25	31	48
Weighted	80	43	36	32
UVD	82	31	28	47
Ours	**91**	**24**	**16**	**62**

(b) Subgoal alignment

Table 1: **Human evaluation of annotations** (% of judgments). (a) Plans with missing steps, too granular, too broad, or free of all three issues (clean). (b) Subgoals that name the correct object and action, show the action only partially, include extra actions, or are fully aligned with the clip.

System 1 input	Overall	Atomic-seen	Comp.-seen	Comp.-unseen
Full context	**29.5**	56.9	14.7	**13.4**
w/o anchor image	28.7	**59.2**	**15.3**	7.8
w/o metadata	23.3	46.4	12.2	7.8
w/o robot state	24.8	53.3	10.9	6.6
w/o task goal	21.6	43.6	10.6	7.8
estimated length ($\hat{L}$)	26.1	56.4	9.7	8.4

Table 2: **System 1 ablations** (%) under our open-loop protocol (Appendix A.3). Each row removes one input; the last row replaces the estimated length $\hat{L}$ from System 2 with elapsed steps.

4.1 Annotation Quality

We evaluate annotation quality through a Prolific [1] study covering 30 episodes from RoboCOIN [26], LBM [27], and RoboCasa [28] (10 each). In total, 269 participants provided 3,447 responses, with three judgments per item (see Appendix A.1.1). We compare four strategies for preparing visual inputs for Gemini-3.1-Pro-based annotation. Uniform sampling selects up to 20 evenly spaced frames from each video, while weighted sampling selects keyframes based on gripper motion speed, sampling more densely during faster movements. UVD uses only visual-semantic boundaries to segment the video. Our method combines UVD boundaries with gripper open/close transitions and end-effector motion cues. Compared with these alternatives, our method produces the highest proportions of clean milestone plans (61% vs. 49-53%) and fully aligned subgoals (62% vs. 32-48%; Table 1), while reducing missing plan steps and extraneous subgoal actions.

4.2 Model Selection and Experimental Setup

We first determine which inputs System 1 needs for reliable execution and which backbone and fine-tuning strategy best support System 2's reasoning modes. To limit the cost of configuration selection, we conduct System 1 input ablations using 10% of the training data (full training set contain a total 33M samples) before full-scale training. For System 2, we compare fully trained models on a validation set.

System 1 input design. We train each configuration on 3.3M samples and evaluate it under an open-loop protocol: System 1 executes annotated subgoals sequentially, without System 2, and is scored by episode success. Providing annotated subgoals isolates the effect of low-level input design from high-level planning errors. As shown in Table 2, the full configuration achieves the highest overall success of 29.5%. Removing the task goal, execution metadata, or robot state reduces success by 7.9, 6.2, and 4.7 percentage points, respectively. The anchor image improves composite-unseen success from 7.8% to 13.4%, while replacing estimated subgoal length with elapsed steps reduces overall success from 29.5% to 26.1%. We therefore retain all inputs for full-scale System 1 training.

[1] https://www.prolific.com/

Backbone	Tuning	Plan↑	Subgoal↑	Judge↑	Est. len.↑	Thought↑	Narration↑	Summary↑	No-video drop
Qwen3.5-4B	Full	65.3	86.0	89.9	57.2	.404	.326	.416	−10.1
Qwen3.5-4B	LoRA	65.5	87.0	89.9	53.5	.407	.349	**.423**	−3.5
Qwen3-VL-4B	Full	**70.5**	**92.5**	**95.7**	61.7	.421	**.464**	.416	−9.1
Qwen3-VL-4B	LoRA	70.3	92.4	95.5	**63.1**	**.422**	.458	.422	−8.2

Table 3: **System 2 ablations** (offline, held-out target-split data). Plan and Subgoal are LLM-judge scores (%); Judge and Est. len. are exact-match accuracies (%); Thought, Narration, and Summary are BLEU. No-video drop measures the performance change when visual input is removed and predictions rely on textual context alone.

Policy	Overall (50)	Atomic-seen (18)	Comp.-seen (16)	Comp.-unseen (16)
Xiaomi-Robotics-1 [29] []	**57.4**	**80.2**	**57.1**	**32.1**
ABot-M0.6 [30]	46.6	<u>79.4</u>	48.3	7.9
ABot-M0.5 [30]	40.3	75.6	37.7	3.3
$\pi_{0.5}$ [2]	16.9	39.6	7.1	1.2
MIRA w/o anchor	39.2	68.7	32.1	13.1
MIRA	45.5	71.9	42.7	18.5
MIRA + steering	<u>53.7</u>	<u>79.4</u>	<u>54.4</u>	<u>24.0</u>

Table 4: **Closed-loop task success rates (%) on RoboCasa365.** Parentheses indicate task counts. Top: public leaderboard results; bottom: $\pi_{0.5}$ baseline and MIRA variants. "+ steering" applies the rules in Sec. 4.4. Best results are **bolded** and second-best results are <u>underlined</u> (including ties).

System 2 backbone selection. We compare Qwen3-VL-4B and Qwen3.5-4B under full fine-tuning and LoRA, using 1.1M multimodal examples spanning all three modes. Validation results in Table 3 show that fully fine-tuned Qwen3-VL-4B performs best on planning (70.5%), subgoal prediction (92.5%), execution judgment (95.7%), and narration (0.464 BLEU). Removing video reduces performance by 3.5–10.1 percentage points, indicating that visual evidence contributes substantially to execution-aware reasoning beyond the available language context. These results motivate our choice of fully fine-tuned Qwen3-VL-4B with video inputs as System 2.

Full-scale training and evaluation. With both configurations selected, we then train the final MIRA system on full RoboCasa365 pretrain split, and evaluate it on the target split [28], comprising 50 tasks: 18 atomic-seen, 16 composite-seen, and 16 composite-unseen. We evaluate 30 episodes per task (1,500 in total) and report closed-loop success rates, comparing with the leaderboard results.

4.3 RQ1: Closed-Loop Performance

As shown in Table 4, MIRA achieves 45.5% overall success, compared with 16.9% for the released $\pi_{0.5}$ baseline (a gain of 28.6 percentage points and approximately 2.7 times the success rate). Despite the limited standalone performance of $\pi_{0.5}$ on RoboCasa365, integrating it into our mixed-initiative hierarchy yields performance comparable to or better than Xiaomi-Robotics-1 [29] and ABot [30], which are trained on substantially more robot data. Starting from the released checkpoint, we train MIRA using only RoboCasa demonstrations and their automatically generated annotations, without additional large-scale external pretraining. These results highlight the substantial gains achievable by equipping an existing policy with progress awareness, high-level planning, and execution monitoring. Success increases from 7.1% to 42.7% on composite-seen tasks and from 1.2% to 18.5% on composite-unseen tasks, demonstrating the hierarchy's effectiveness on multi-stage tasks and unfamiliar task compositions. Removing the anchor image reduces overall success from 45.5% to 39.2%, indicating its role in supporting execution. Figure 4 illustrates how System 1 tracks progress and initiates handoffs to System 2 for execution assessment and plan updates.

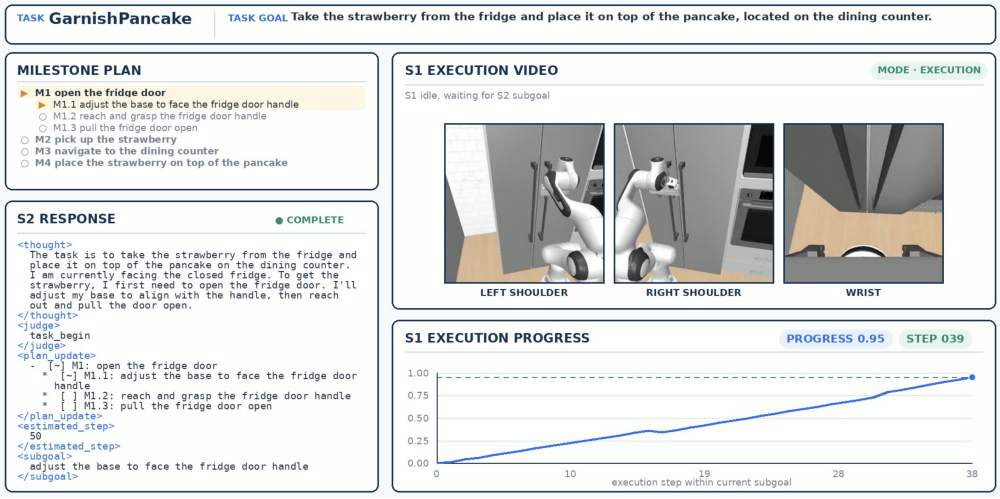


Figure 4: **Closed-loop rollout of MIRA on *GarnishPancake*.** *Left:* The milestone plan, active subgoal, and structured System 2 response, including its execution assessment and updated subgoal. *Right:* Multi-view observations and the subgoal progress predicted by System 1. When the handoff condition is met, System 2 assesses the preceding execution clip before deciding how to proceed.

Language steering rule	Overall	Atomic-seen	Comp.-seen	Comp.-unseen
None (MIRA)	45.5	71.9	42.7	18.5
+ regrasp	46.5	72.2	44.2	19.8
+ press again	46.5	72.6	43.5	20.0
+ retry last milestone	51.1	79.1	44.0	**26.7**
+ all	**53.7**	**79.4**	**54.4**	24.0

Table 5: **Language steering** (success rate, %). Each intermediate row applies one rule independently to MIRA; the final row combines all rules. Steering edits plans or subgoals, not continuous actions.

4.4 RQ2: Test-time Language Steering

As shown in Table 4, language steering further improves MIRA's overall success from 45.5% to 53.7% without retraining either model, achieving more than three times the success rate of the released $\pi_{0.5}$ baseline. The largest gain occurs on composite-seen tasks, where success increases from 42.7% to 54.4%, demonstrating that language-level interventions can resolve additional execution failures within the existing hierarchy.

We steer execution through simple corrections to high-level plans and subgoals, such as regrasping an object after a failed grasp, repeating an button press for several more times, retrying the previous milestone if the rollout is neither timeout nor successful. These interventions rely on heuristic rules applied to available observations, without access to privileged state information. Table 5 evaluates the contribution of each rule. Retrying the previous milestone provides the largest individual gain, raising overall success from 45.5% to 51.1% and composite-unseen success from 18.5% to 26.7%. Combining all rules achieves the highest overall success of 53.7%. These results highlight the practical value of the language interface, which enables execution corrections by revising what the robot should do next, without modifying continuous actions or model parameters.

4.5 RQ3: One-Shot Learning from a Single Video

We evaluate whether System 2 can distill a single successful demonstration video into a reusable recipe for each composite-unseen task. The original instructions already specify object locations and intermediate steps, leaving little procedural information to infer from the demonstration. To

Task specification	System 2 modes	Comp.-unseen
Short goal	Plan + Exec.	3.5
Short goal + recipe	Plan + Exec. + Summary	9.6
Short goal + recipe + language steering	Plan + Exec. + Summary	**16.7**
Full goal (reference)	Plan + Exec.	18.5

Table 6: **One-shot learning from a single video** on composite-unseen tasks (success rate, %). Summary Mode distills one demonstration per task into a recipe supplied to Plan Mode; System 1 receives no demonstration data. The full goal is shown for reference.

assess Summary Mode's ability to extract this information, we reformulate the setting as one-shot learning: the model receives one demonstration video and a short task goal (e.g., *"garnish the pancake"*), with detailed instructions withheld (e.g., *"take the strawberry from the fridge and place it on top of the pancake, located on the dining counter."*). Summary Mode extracts a recipe from the video, which Plan Mode then uses to guide execution. System 1 receives no demonstration, and neither model is updated.

As shown in Table 6, the recipe raises success from 3.5% to 9.6%, and language steering further improves it to 16.7%, approaching the 18.5% achieved with full instructions. The *GarnishPancake* recipe, for example, recovers the required sequence from fridge opening to strawberry placement (Appendix A.2.4). These results demonstrate that a single video can provide actionable procedural memory without gradient updates.

5 Conclusion

We presented MIRA, a mixed-initiative framework that integrates reasoning, action, and memory for long-horizon robot manipulation. System 1 tracks execution progress and determines *when to think*, while System 2 plans *what to do next*, monitors execution, and distills demonstrations into reusable procedural memory. On RoboCasa365, MIRA raises $\pi_{0.5}$'s success rate from 16.9% to 45.5%, and language steering further improves it to 53.7%. A single demonstration also improves performance on unseen task compositions by supplying procedural knowledge. These findings support a hierarchy in which reasoning guides action and execution informs deliberation. Future work will use execution feedback to jointly improve both systems, allowing robots to refine their planning, control, and memory through experience.

References

[1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. Openvla: An open-source vision-language-action model. In *Proceedings of the Conference on Robot Learning*, 2025.

[2] K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of the Conference on Robot Learning*, 2025.

[3] L. X. Shi, B. Ichter, M. R. Equi, L. Ke, K. Pertsch, Q. Vuong, J. Tanner, A. Walling, H. Wang, N. Fusai, et al. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. In *Proceedings of the International Conference on Machine Learning*, 2025.

[4] S. Han, B. Qiu, Y. Liao, S. Huang, C. Gao, S. Yan, and S. Liu. Robocerebra: A large-scale benchmark for long-horizon robotic manipulation evaluation. *arXiv preprint arXiv:2506.06677*, 2025.

[5] M. Torne, K. Pertsch, H. Walke, K. Vedder, S. Nair, B. Ichter, A. Z. Ren, H. Wang, J. Tang, K. Stachowicz, K. Dhabalia, M. Equi, Q. Vuong, J. T. Springenberg, S. Levine, C. Finn, and D. Driess. Mem: Multi-scale embodied memory for vision language action models. *arXiv preprint arXiv:2603.03596*, 2026.

[6] Physical Intelligence. $\pi_{0.7}$: a steerable generalist robotic foundation model with emergent capabilities. *arXiv preprint arXiv:2604.15483*, 2026.

[7] I. Liu, A.-C. Cheng, R. Yan, G. Chen, R.-Z. Qiu, X. Zou, S. Yi, H. Yin, X. Wang, and S. Liu. Long-horizon manipulation via trace-conditioned vla planning. *arXiv preprint arXiv:2604.21924*, 2026.

[8] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. In *Proceedings of the Conference on Robot Learning*, 2023.

[9] J. Hu, M. Shridhar, C. Lu, D. Shah, H.-T. L. Chiang, J. Tan, and A. Xie. What matters in orchestrating robot policies: A systematic study of hierarchical vla agents. *arXiv preprint arXiv:2606.10267*, 2026.

[10] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, W. Ding, C. Gao, C. Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.

[11] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on Robot Learning*, 2022.

[12] S. Belkhale, T. Ding, T. Xiao, P. Sermanet, Q. Vuong, J. Tompson, Y. Chebotar, D. Dwibedi, and D. Sadigh. Rt-h: Action hierarchies using language. *arXiv preprint arXiv:2403.01823*, 2024.

[13] Y. Dai, J. Lee, N. Fazeli, and J. Chai. Racer: Rich language-guided failure recovery policies for imitation learning. In *2025 IEEE international conference on robotics and automation (ICRA)*, pages 15657–15664. IEEE, 2025.

[14] H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, T. Wang, E. Zhou, H. Fan, X. Zhang, and G. Huang. Memoryvla: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. *International Conference on Learning Representations*, 2026.

[15] A. Sridhar, J. Pan, S. Sharma, and C. Finn. Memer: Scaling up memory for robot control via experience retrieval. In *International Conference on Learning Representations*, 2026.

[16] Y. Dai, H. Fu, J. Lee, Y. Liu, H. Zhang, J. Yang, C. Finn, N. Fazeli, and J. Chai. Robomme: Benchmarking and understanding memory for robotic generalist policies. *arXiv preprint arXiv:2603.04639*, 2026.

[17] K. Sridhar, S. Dutta, D. Jayaraman, and I. Lee. Ricl: Adding in-context adaptability to pre-trained vision-language-action models. In *Proceedings of the Conference on Robot Learning*, 2025.

[18] G. Chen, M. Wang, Q. Shao, Z. Zhou, W. Mao, T. Cui, M. Zhu, Y. Deng, L. Yang, Z. Zhang, Y. Yang, H. Chen, and Y. Yue. See once, then act: Vision-language-action model with task learning from one-shot video demonstrations. *arXiv preprint arXiv:2512.07582*, 2025.

[19] Generalist Team. Gen-1.5: Embodied foundation models are one-shot learners. *Generalist AI Blog*, 2026. August 19, 2026.

[20] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox. Mimicgen: A data generation system for scalable robot learning using human demonstrations. In *7th Annual Conference on Robot Learning*, 2023. URL https://arxiv.org/abs/2310.17596.

[21] Z. Zhang, Y. Li, O. Bastani, A. Gupta, D. Jayaraman, Y. J. Ma, and L. Weihs. Universal visual decomposer: Long-horizon manipulation made easy. In *IEEE International Conference on Robotics and Automation*, 2024.

[22] L. Kou, F. Ni, Y. Zheng, P. Han, J. Liu, H. Cui, R. Liu, and J. Hao. Roboannotatorx: A comprehensive and universal annotation framework for accurate understanding of long-horizon robot demonstration. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10353–10363, 2025. URL https://openaccess.thecvf.com/content/ICCV2025/html/Kou_RoboAnnotatorX_A_Comprehensive_and_Universal_Annotation_Framework_for_Accurate_Understanding_ICCV_2025_paper.html.

[23] H. Li, Z. Wang, Z.-h. Ding, S. Yang, Y. Chen, Y. Tian, X. Hu, T. Wang, D. Lin, F. Zhao, S. Liu, and J. Pang. Robointer: A holistic intermediate representation suite towards robotic manipulation. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/b81d83165e3145a2e7d33bb5e33ea913-Abstract-Conference.html.

[24] J. Liu, H. Chi, L. Zhang, Y. Xie, Y. Wang, L. Chen, H. Ye, X. Hao, and W. Ding. Thinking in text and images: Interleaved vision–language reasoning traces for long-horizon robot manipulation. *arXiv preprint arXiv:2605.00438*, 2026. URL https://arxiv.org/abs/2605.00438.

[25] J. Peng, X. Yu, D. Feng, Y. Yang, W. Cai, J. Xiong, M. Y. Yang, Galnliker, D. Ghosh, L. Groom, Hausmn, J. Zheng, J. Cao, X. Wei, K. Pang, B. Ichter, et al. Cortex: A bidirectionally aligned embodied agent framework for long-horizon manipulation. *arXiv preprint arXiv:2607.05377*, 2025Jiangmiao and Shen, Yuan and Wang, Tai. URL https://arxiv.org/abs/2607.05377.

[26] S. Wu, X. Liu, S. Xie, P. Wang, X. Li, B. Yang, Z. Li, K. Zhu, H. Wu, Y. Liu, et al. Robocoin: An open-sourced bimanual robotic data collection for integrated manipulation. *arXiv preprint arXiv:2511.17441*, 2025.

[27] J. Barreiros, A. Beaulieu, A. Bhat, R. Cory, E. Cousineau, H. Dai, C.-H. Fang, K. Hashimoto, M. Z. Irshad, M. Itkina, et al. A careful examination of large behavior models for multitask dexterous manipulation. *Science Robotics*, 11(113):eaea6201, 2026.

[28] S. Nasiriany, S. Nasiriany, A. Maddukuri, and Y. Zhu. Robocasa365: A large-scale simulation framework for training and benchmarking generalist robots. *arXiv preprint arXiv:2603.04356*, 2026.

[29] X. R. Team, J. Guo, P. Jin, J. Li, P. Li, Y. Li, F. Liu, W. Peng, O. Qin, Y. Su, et al. Xiaomi-robotics-1: Scaling vision-language-action models with over 100k hours of real-world trajectories. *arXiv preprint arXiv:2607.15330*, 2026.

[30] R. Chen, Y. Yang, Z. Tang, D. Huo, T. Lin, H. Wu, H. Liu, Y. Chen, L. Zheng, B. Yuan, T. Li, M. Wang, D. Qi, B. Hu, W. Mei, Y. Xuan, H. Yang, Y. Zhu, M. Xu, Z. Ma, and X. Chang. Abot-m0.5: Unified mobility-and-manipulation world action model. *arXiv preprint arXiv:2607.00678*, 2026.

Appendix

A.1 Trajectory Segmentation and Annotation

Notation. We represent a trajectory as $\tau = \{(o_t, \mathbf{x}_t, c_t)\}_{t=0}^{T-1}$, where o_t denotes camera observations, $\mathbf{x}_t$ the end-effector (EE) poses, and c_t the gripper apertures across arms. We segment each trajectory using complementary state and semantic cues, with each boundary marking the start of a new clip.

State boundaries. Gripper open/close transitions define coarse boundaries $\mathcal{B}_{\mathrm{c}}$. Within each coarse segment, changes in EE speed $\|\dot{\mathbf{x}}_t\|$ define fine boundaries $\mathcal{B}_{\mathrm{f}}$ between moving and near-stationary phases.

Semantic boundaries. To complement these kinematic cues, we apply Universal Visual Decomposer (UVD) [21] within each coarse segment to obtain semantic boundaries $\mathcal{B}_{\mathrm{s}}$. Taking the final frame as its initial goal, UVD searches backward for the earliest frame from which the visual-embedding distance to the goal decreases monotonically. It then adopts that frame as the next goal and repeats the search, producing boundaries based on visual features.

Merging and labeling. We take the union $\mathcal{B}_{\mathrm{c}} \cup \mathcal{B}_{\mathrm{f}} \cup \mathcal{B}_{\mathrm{s}}$ and merge boundaries within a temporal tolerance δ. Given the resulting clips, task instruction, initial scene, and state-derived context, Gemini 3.1 Pro selects the final boundaries and generates an initial-scene description, a milestone plan, clip-level subgoals, and narrations. Each milestone groups consecutive subgoals. For example, "open the box with the left gripper" consists of "reach toward the box flap", "grasp the box flap", "pull the box flap open", and "release the box flap" (Fig. 2).

Progress targets. For a subgoal segment spanning frames $[s_k, e_k)$, we define the progress target as

$$p_t = \frac{t - s_k}{e_k - 1 - s_k}, \qquad s_k \le t < e_k. \tag{2}$$

This target tracks elapsed progress within a demonstrated segment, while System 2 assesses whether the subgoal has actually been completed.

Training data for System 2. These annotations provide supervision for all three modes. Plan Mode learns to generate a milestone plan from the initial scene and task goal. Execution Mode uses the current plan state, execution clip, and goal to predict an execution judgment, plan update, next subgoal, and estimated duration bin. Summary Mode learns to narrate individual clips and summarize their ordered narrations into a task recipe. We will release the post-processed annotations.

A.1.1 Human Study

We evaluate annotation quality on 10 episodes from each of three datasets: RoboCOIN, an internal LBM dataset, and RoboCasa. We compare four boundary-proposal methods—uniform sampling, weighted sampling, UVD alone, and our hybrid approach—and collect 3,447 responses from 269 Prolific participants, with three judgments per item.

For plans, participants assess whether necessary steps are missing, the decomposition is overly granular, or the steps are too broad. A plan is *clean* when none of these issues is identified. For subgoals, participants view the clip, task instruction, and plan, then assess whether the action–object label is correct, the clip covers only part of the labeled action, or extra actions are present. A subgoal is *fully aligned* when its label is correct and the clip covers the complete action without extra actions. Figure 5 shows the subgoal assessment interface.

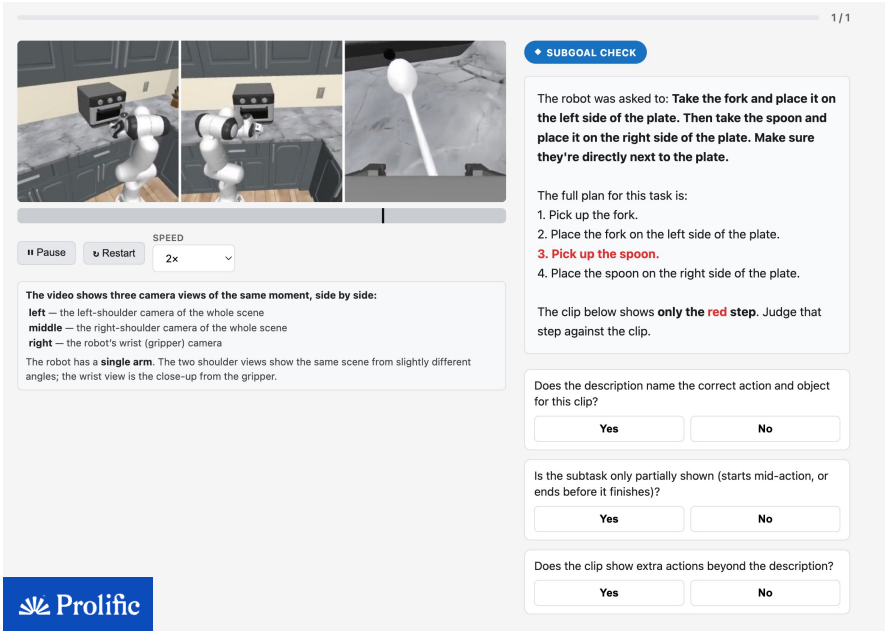


Figure 5: **Human study interface for subgoal assessment.** Participants watch three camera views of a clip (left), read the task instruction and full plan with the evaluated step highlighted in red (right), and answer three yes/no questions about action–object correctness, partial coverage, and extra actions.

A.2 System 2 Details

System 2 operates in Plan, Execution, and Summary Modes using shared model weights and mode-specific system prompts and queries (Fig. 3a). For visual input, we tile the three camera views into a single image or video. The following subsections describe the modes, their use of recipe memory, and the training and offline evaluation setup.

A.2.1 Plan Mode

Plan Mode takes the task goal and initial camera observations and produces a milestone checklist. Its output begins with scene analysis and reasoning in `<thought>`, followed by the checklist in `<plan>`. Milestones are marked as completed `[x]`, in progress `[~]`, or pending `[ ]`. Figure 6 shows an example for "pick the wooden spoon from the drawer and place it on the counter."

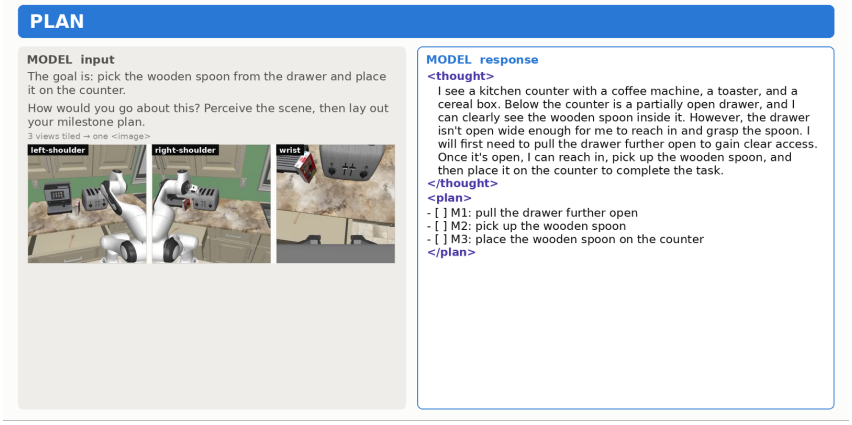


Figure 6: **Plan Mode.** Given the task goal and initial observations, System 2 analyzes the scene and generates a three-milestone plan for retrieving a wooden spoon from a drawer and placing it on the counter.

A.2.2 Execution Mode

At each handoff, Execution Mode uses the task goal, current plan state, and latest execution clip to assess the preceding segment, update the milestone checklist, and select the next subgoal z_j with estimated duration $\hat{L}_j$. It returns `<thought>`, `<judge>`, `<plan_update>`, `<estimated_step>`, and `<subgoal>`, optionally followed by `<subgoal_detail>` for a more detailed instruction. Thus, each response both assesses the preceding execution and specifies what System 1 should do next.

The judgment is one of five labels: `task_begin`, `subgoal_completed`, `subgoal_incomplete`, `subgoal_failed`, or `task_finish`. Duration is measured in control steps and selected from 16 bins: 50, 75, 100, 125, 150, 175, 200, 250, 300, 350, 400, 500, 600, 800, 1000, and 1300. Figure 7 illustrates the transition after a completed subgoal.

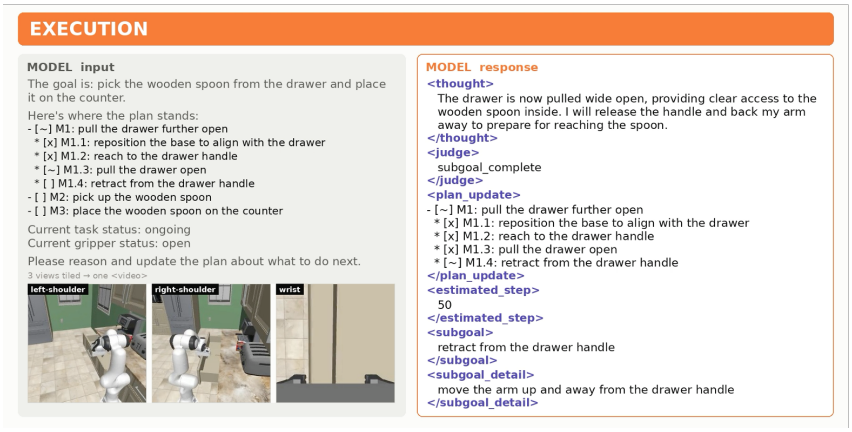


Figure 7: **Execution Mode.** Given the plan state and latest execution clip, System 2 judges the preceding subgoal complete, checks off M1.3, and issues the next subgoal with an estimated duration. The task is the same as in Fig. 6.

A.2.3 Summary Mode

Summary Mode distills a demonstration into a reusable task recipe by first narrating its clips and then summarizing the resulting narrations.

Clip narration. System 2 processes demonstration clips in order, conditioning each narration on the task goal, current clip, and earlier narrations. It describes the demonstrated actions in `<narration>`, retaining context across clips. Figure 8 shows an example.

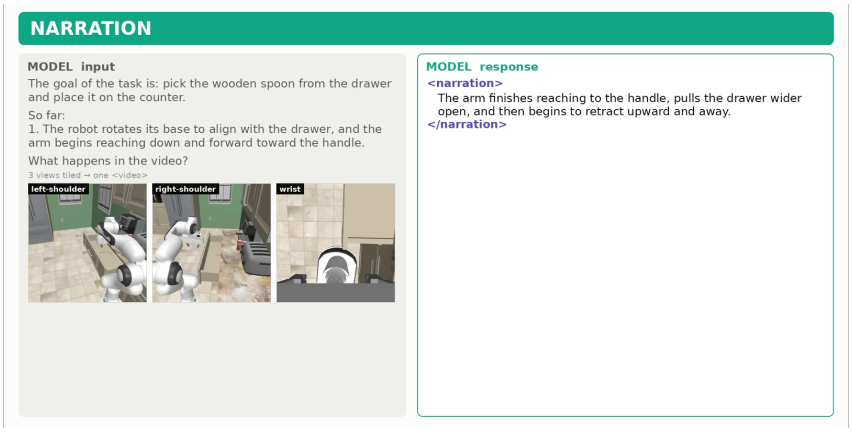


Figure 8: **Summary Mode: clip narration.** Conditioned on the task goal and preceding narrations, System 2 describes a demonstration clip from the task in Fig. 6.

Recipe summarization. Once all clips have been narrated, System 2 condenses the narrations into a procedural recipe in `<summary>` (Fig. 9). This language-based memory can then guide planning without requiring System 1 to process the demonstration video or actions.

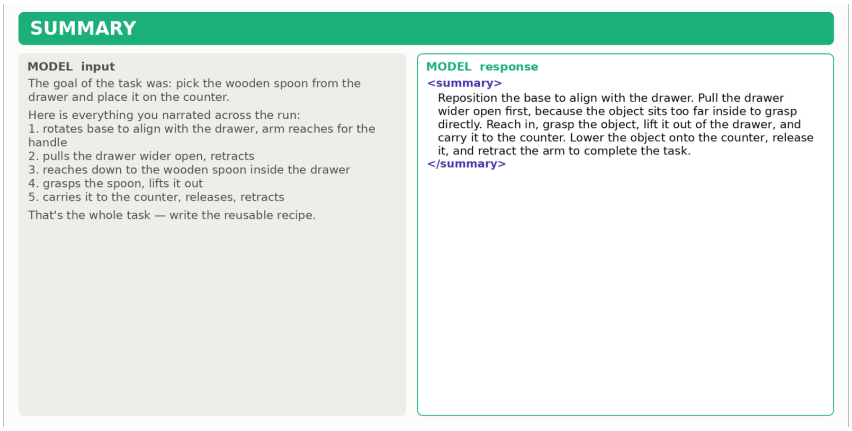


Figure 9: **Summary Mode: recipe summarization.** System 2 condenses the ordered clip narrations into a reusable task recipe. The demonstration is the same as in Fig. 8.

A.2.4 Plan Mode with Recipe Memory

At test time, a recipe distilled from a demonstration can be supplied to Plan Mode alongside the current goal and initial observations. Plan Mode adapts this procedural guidance to the current scene to produce a milestone plan (Fig. 10), which guides execution through the standard hierarchy. This process requires neither parameter updates nor direct reuse of demonstration actions.

One-shot evaluation. Because the original task instructions already specify object locations and intermediate steps, they leave little procedural information to infer from a demonstration. We therefore evaluate Summary Mode in a one-shot setting, replacing the detailed instructions with a short task goal and one successful demonstration video per composite-unseen task. Summary Mode ex-

tracts the missing procedural information into a recipe for Plan Mode. System 1 receives no demonstration, and neither model is updated.

Example: GarnishPancake. For "garnish the pancake", Summary Mode produces the following recipe from a single demonstration: *"Navigate to the refrigerator and pull open the right door to access the items inside. Reach into the refrigerator, grasp the strawberry, and lift it out. Navigate to the kitchen counter to locate the plate of pancakes, then align the strawberry over the stack of pancakes, lower it, and release it to place it on top."* Plan Mode uses this recipe to generate four milestones: M1 open the right refrigerator door; M2 pick up the strawberry; M3 navigate to the plate of pancakes on the counter; and M4 place the strawberry on top of the pancake.

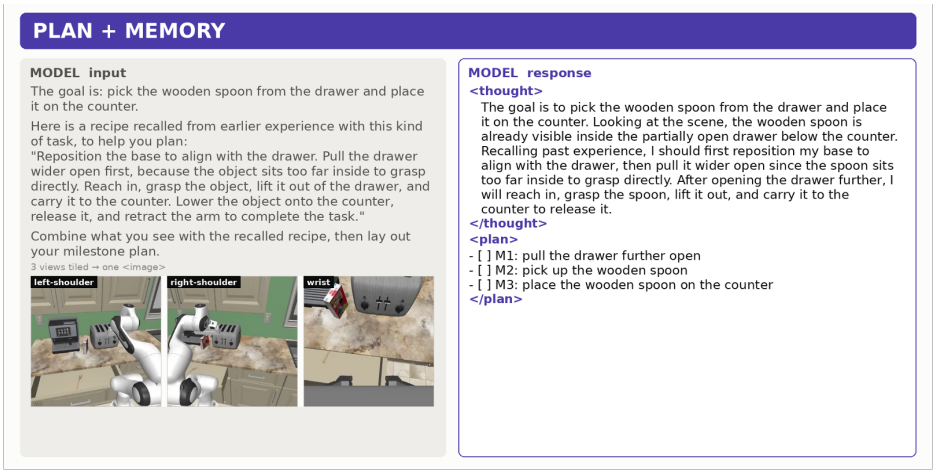


Figure 10: **Plan Mode with recipe memory.** A recipe distilled from a demonstration is added to the planning prompt. System 2 combines it with the current goal and scene to generate a milestone plan for GarnishPancake.

A.2.5 Training and Offline Evaluation

Training. We train System 2 on 1.1M examples for three epochs with a batch size of 192, using learning rates of 1×10^{-5} for the language model and 2×10^{-6} for the vision encoder. We sample videos at 4 FPS, retain at most 32 frames, and use spatial and temporal merge factors of 2. The maximum sequence length is 8,192 tokens.

Offline evaluation. For the System 2 ablations in Table 3, we score plans and subgoals against the annotations using an LLM judge. Judgment labels and duration bins are evaluated by exact string match, while free-form thoughts, narrations, and summaries are evaluated using BLEU.

A.3 System 1 Details

Inputs. At the start of each subgoal, we capture an anchor image and keep it fixed until the next handoff (Fig. 3b). We augment the language prompt with the current subgoal and execution metadata, retaining the task goal and discretizing robot states as in $\pi_{0.5}$. The example below compares the original prompt with the adapted System 1 prompt.

Original $\pi_{0.5}$ prompt	Adapted System 1 prompt
`Task: close the lid blender by securely placing the lid on top; Current State: 120 132 172 250 137 159 136; Action:`	`Task: close the lid blender by securely placing the lid on top; Current Subgoal: reach to the blender lid; Estimated Length: 100; Initial State: 120 132 172 250 137 159 136; Current State: 127 131 171 250 139 122 139; Current Gripper: Open; Action:`

Training. We append a progress scalar to the action target and train it jointly with the standard $\pi_{0.5}$ action objective. Fine-tuning uses 3.3M samples, approximately 10% of the available RoboCasa365 training data, with a batch size of 512 and a fixed learning rate of 5×10^{-5}. Training runs for 270k steps and takes approximately 1.5 days.

Open-loop development protocol. We use an open-loop protocol to evaluate the System 1 ablations in Table 2 independently of System 2. The evaluation uses 1,000 additionally annotated episodes, with 20 episodes per target task.

A.4 Handoff Criteria

System 1 hands control back to System 2 when any of the following conditions is met:

- **Progress saturation:** predicted progress exceeds 0.95 for five consecutive control steps.
- **Motor convergence:** the norm of the EE delta action remains below 10^{-3} for five consecutive steps with no gripper open/close event. This condition avoids a handoff during gripper opening or closing.
- **Adaptive timeout:** execution reaches $\max(400,\, 2\hat{L})$ steps since the subgoal began, allowing twice the estimated duration with a minimum of 400 steps.

A.5 Language Steering Rules

Language steering revises high-level plans and subgoals to correct execution. Interventions are triggered by heuristic rules applied to available observations and System 2's assessment, without privileged state information. Continuous actions and model parameters remain unchanged. We use four types of intervention:

- **Regrasp:** when a grasp is judged to have failed, issue a new grasp subgoal for the same object.
- **Press again:** when the outcome of a button press or control activation is uncertain, repeat the activation.
- **Retry last milestone:** reattempt the most recent milestone.
- **Task-specific hints:** provide a small set of manually written hints for individual tasks.

A.6 Benchmark Details

RoboCasa365 [28] simulates a Franka Panda arm mounted on an Omron mobile base in kitchen environments, with observations from left-shoulder, right-shoulder, and wrist cameras. We use two splits:

- **Pretraining split:** 300 tasks—65 atomic and 235 composite—with approximately 30k demonstrations totaling over 400 hours.
- **Target split:** 50 tasks—18 atomic-seen, 16 composite-seen, and 16 composite-unseen—with over 25k released demonstrations totaling over 300 hours. This split is used for our main evaluation.